Overview of Operating Systems and Kernels

An Operating System (OS) is the *master controller* of a computer. It hides hardware complexity and provides a clean, easy-to-use interface for programs and users.

It's a system software that acts as an intermediatory/interface between users and underlying hardware.

Functions of OS

Process Management

- A *process* is a running program.
- The kernel creates, schedules, suspends, resumes, and terminates processes.
- It ensures every process gets CPU time fairly and efficiently.

Memory Management

- Each process gets its own protected memory space (virtual memory).
- OS maps virtual addresses to physical RAM using structures like **page tables**.
- It handles allocation, swapping, caching — making sure memory is never misused.

File System Management

- The OS stores and retrieves data using file systems (ext4, XFS, etc.).
- Provides abstraction: *everything is a file* — even devices.

Device and I/O Management

- Devices are accessed via unified interfaces.
- Kernel drivers translate OS requests to hardware operations.

Networking

- The kernel implements low-level network stack protocols (TCP/IP).
- User apps see simple sockets while kernel handles all complexity.

Inter-Process Communication (IPC)

- Pipes, message queues, shared memory, signals — for processes to talk to each other smoothly.

Security & Protection

- Permissions, user IDs, capabilities — ensure processes cannot harm each other.
- Kernel enforces isolation and controlled access.

Kernel

The **Kernel** is the *core* of the operating system — it runs in **privileged mode** and controls **everything**.

It is the portion of the OS that executes in kernel mode and manages hardware resources directly.

The user interface is the outer most portion of the operating system, the kernel is the innermost.

The kernel is sometimes referred to as the supervisor, core, or internals of the operating system.

Only the kernel can access hardware directly.

Typical components of a kernel are interrupt handlers to service interrupt requests, a scheduler to share processor time among multiple processes, a memory management system to manage process address spaces, and system services such as networking and inter process communication.

On modern systems with protected memory management units, the kernel typically resides in an elevated system state compared to normal user applications. This includes a protected memory space and full access to the hardware. This system state and memory space is collectively referred to as kernel-space.

When executing kernel code, the system is in kernel-space executing in kernel mode. When running a regular process, the system is in user-space executing in user mode.

Applications running on the system communicate with the kernel via system calls.

A **system call** is a *request from a user program to the operating system* to do something that the program **cannot do directly**.

User programs are *not allowed* to access hardware or kernel resources, so they must ask the kernel for help.

Examples of system calls:

- Read a file
- Write to the screen
- Open a file
- Create a process
- Allocate memory

Example : What happens when you write a *printf* statement?

We know that printf() is a C (usually glibc-GNU C Library for LINUX - It connects user programs to the Linux kernel)) library function, not a kernel function . So, it does not directly talk to the kernel. It prepares the text to print and store the output into a user space buffer.

printf() internally calls the write (fd , buffer, size) system call (glibc has system call wrappers - small functions acts as a middle man between your program and kernel)

where fd =1 for stdout, buffer is your text and size is the number of bytes.

Because only the kernel can actually print to the screen.

So printf() → asks the kernel:

"Please write these characters to the screen."

As its a **system call**, and the CPU enters kernel mode (privileged mode).

The kernel sends the characters to the terminal driver, which handles the screen output.

After finishing the printing, the kernel gives control back to your program.ie, it gets back to the user mode from kernel mode.

A **privilege change** means the CPU switches between **two different power levels**:

- **User mode (low privilege)**
- **Kernel mode (high privilege)**

These are often called **rings**:

- User mode = **Ring 3**
- Kernel mode = **Ring 0**

If user programs had full power:

- They could access any memory
- They could crash or overwrite the kernel
- They could control hardware directly
- They could break the whole system

So, the CPU protects the system by restricting what user code can do.

Privilege switches happen ONLY in three cases:

1. System Call

These instructions **tell the CPU**:

"Switch to kernel mode and jump to the system call handler."

The CPU:

- Changes to **Ring 0**
- Loads kernel stack
- Jumps to kernel code

2.Hardware Interrupt

Keyboard, mouse, network, disk → interrupt CPU.

CPU immediately:

- Pauses user process
- Switches to kernel mode
- Runs interrupt handler

3. Exceptions

Divide by zero, page fault, invalid memory access.

CPU:

- Switches to kernel mode
- Calls exception handler

When kernel finishes its work, it returns to user mode.

In Linux, we can generalize that each processor is doing exactly one of three things at any given moment:

- In user-space, executing user code in a process
- In kernel-space,(in process context), executing on behalf of a specific process
- In kernel-space,(in interrupt context), not associated with a process, handling an interrupt

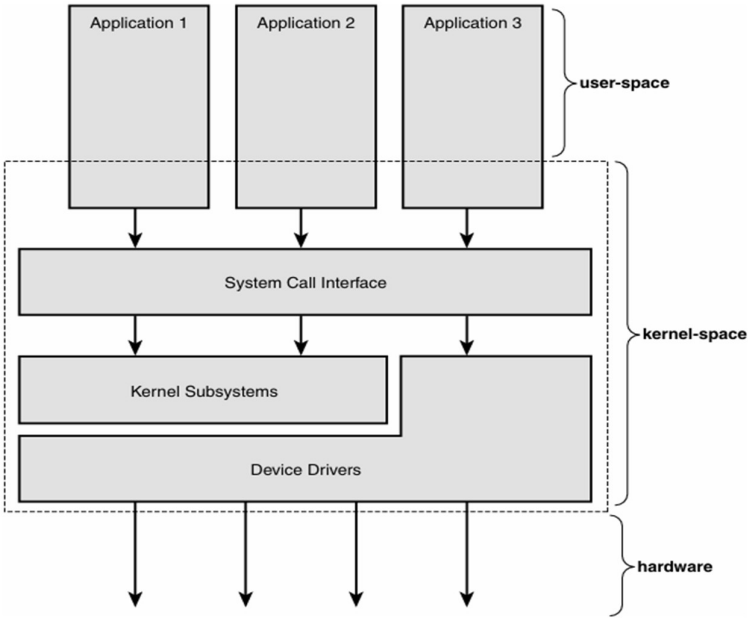


Figure 1.1 Relationship between applications, the kernel, and hardware.

Linux vs Classic UNIX models

A Unix kernel is typically a monolithic static binary.

That is, it exists as a single, large, executable image that runs in a single address space.

Unix systems typically require a system with a paged memory-management unit (MMU); this hardware enables the system to enforce memory protection and to provide a unique virtual address space to each process.

Monolithic Kernel Versus Microkernel Designs

Monolithic kernels are the simpler design of the two, and all kernels were designed in this manner until the 1980s.

Monolithic kernels are implemented entirely as a single process running in a single address space.

Consequently, such kernels typically exist on disk as single static binaries. All kernel services exist and execute in the large kernel address space. Communication within the kernel is trivial because everything runs in kernel mode in the same address space: The kernel can invoke functions directly, as a user-space application might.

Most Unix systems are monolithic in design.

Microkernels, on the other hand, are not implemented as a single large process. Instead, the functionality of the kernel is broken down into separate processes, usually called servers.

Ideally, only the servers absolutely requiring such capabilities run in a privileged execution mode. The rest of the servers run in user-space.

All the servers, though, are separated into different address spaces. Therefore, direct function invocation as in monolithic kernels is not possible. Instead, microkernels communicate via message passing: An inter process communication (IPC) mechanism is built into the system, and the various servers communicate with and invoke “services” from each other by sending messages over the IPC mechanism.

The separation of the various servers prevents a failure in one server from bringing down another. Likewise, the modularity of the system enables one server to be swapped out for another. Because the IPC mechanism involves quite a bit more overhead than a trivial function call, however, and because a context switch from kernel-space to user-space or vice versa is often involved, message passing includes a latency and throughput hit not seen on mono lithic kernels with simple function invocation.

Consequently, all practical microkernel-based systems now place most or all the servers in kernel-space, to remove the overhead of frequent context switches and potentially enable direct function invocation.

The Windows NT kernel (on which Windows XP, Vista, and 7 are based) and Mach (on which part of Mac OS X is based) are examples of microkernels.

Linux is a monolithic kernel; that is, the Linux kernel executes in a single address space entirely in kernel mode.

Linux, however, borrows much of the good from microkernels: Linux boasts a modular design, the capability to preempt itself (called kernel pre emption), support for kernel threads, and the capability to dynamically load separate binaries (kernel modules) into the kernel image.

- **UNIX**
    - Created in the 1970s at AT&T Bell Labs.
    - Proprietary; different vendors created their own versions (AIX, HP-UX, Solaris).
- **Linux**
    - Started by Linus Torvalds in 1991.
    - **Open source (GPL)** and community-developed.

Major Differences between the Linux kernel and classic Unix systems

- Linux supports the dynamic loading of kernel modules. Although the Linux kernel is monolithic, it can dynamically load and unload kernel code on demand.
- Linux has symmetrical multiprocessor (SMP) support. Although most commercial variants of Unix now support SMP, most traditional Unix implementations did not.
- The Linux kernel is preemptive. Unlike traditional Unix variants, the Linux kernel can preempt a task even as it executes in the kernel. Of the other commercial Unix implementations, Solaris and IRIX have preemptive kernels, but most Unix kernels are not preemptive.
- Linux takes an interesting approach to thread support: It does not differentiate between threads and normal processes. To the kernel, all processes are the same— some just happen to share resources.
- Linux provides an object-oriented device model with device classes, hot-pluggable events, and a user-space device filesystem (sysfs).
- Linux ignores some common Unix features that the kernel developers consider poorly designed, such as STREAMS, or standards that are impossible to cleanly implement.
- Linux is free in every sense of the word. The feature set Linux implements is the result of the freedom of Linux's open development model. If a feature is without merit or poorly thought out, Linux developers are under no obligation to implement it.
  To the contrary, Linux has adopted an elitist attitude toward changes: Modifications must solve a specific real-world problem, derive from a clean design, and have a solid implementation. Consequently, features of some other modern Unix variants that are more marketing bullet or one-off requests, such as pageable kernel memory, have received no consideration.

Aspect	Classic UNIX Kernels	Linux Kernel
Code Ownership / Philosophy	Proprietary and closed; controlled by commercial vendors.	Open source; GPL-licensed; community-built.
Kernel Architecture	Monolithic but non-modular; static design; many vendor-specific variations.	Monolithic **but strongly modular** with loadable kernel modules.
Portability	Not written for portability; closely tied to specific hardware platforms.	Designed for portability from the beginning; runs on large variety of hardware.
Development Model	Slow, centralized, vendor-driven updates; patches not shared across systems.	Fast, distributed development through open collaboration and mailing lists.
Design Goals	Stability and vendor consistency were key; innovation depended on a single company.	Flexibility, performance, modularity, and simplicity in design are emphasized.
System Interface	Many systems follow the traditional UNIX interface but differ across vendors.	POSIX-like interface; relies on GNU libraries; strives for consistency.
Modularity & Extensibility	Limited extensibility; kernel changes often required rebuilding the kernel.	Kernel modules can be inserted/removed at runtime; highly extensible.
Hardware Support	Limited to hardware supported by the vendor; often narrow range.	Large, constantly growing hardware support from global contributors.
File System Abstraction	Traditional UNIX filesystem layers; implementation varies per vendor.	Unified **VFS (Virtual File System)** layer giving a single interface for all filesystems.
Scheduling & Performance	Classical Unix schedulers; slower evolution due to proprietary nature.	Modern schedulers developed openly; frequent improvements and tuning.
Process & Memory Management	Traditional algorithms; fewer major revisions over time.	Aggressive performance improvements, optimized MM, updated algorithms.
Community / Contribution	Closed; improvements stay inside vendor boundaries.	Anyone can contribute; massive collaborative development emphasized by Love.

Feature	Classic UNIX Kernels	Linux Kernel
SMP (Symmetric Multiprocessing)	Limited or coarse SMP support;	Designed for strong SMP:.
Kernel Preemption	Typically **non-preemptive** kernel; kernel code runs until completion or voluntary yield.	Fully **preemptive kernel** (except in critical regions
Thread Support	Threads often implemented in **user space** (green threads); not first-class kernel objects.	Threads = lightweight processes created using **clone()**. Kernel treats threads as processes sharing resources.
Device Model	Flat, ad-hoc, vendor-specific device structure; no unified abstraction.	Fully **object-oriented device model**: devices, drivers, buses, and classes represented as structured kernel objects.
Sysfs	Not available. Device info scattered across driver-specific interfaces.	Introduced in Linux 2.6. **sysfs** exposes kernel objects as a filesystem. Provides clean, hierarchical view of devices/drivers.
STREAMS	Used heavily in System V for network and tty I/O. Core part of UNIX I/O pipeline.	Linux **does not use STREAMS**; rejected due to complexity and performance overhead. Uses simpler, faster I/O subsystems.
Kernel Modularity	Limited or no runtime module loading. Kernels were mostly static.	Supports **loadable kernel modules**. Drivers can be added/removed without rebuilding the kernel.
Virtual File System (VFS)	Traditional UNIX FS structures; design varies per vendor.	Unified **VFS layer** offering a clean abstraction for all filesystems. Modern, flexible design.
Networking Stack	STREAMS-based or vendor-specific networking.	High-performance, custom networking stack; modular, fast, widely used in servers and routers.
Process & Memory Scalability	Slower evolution due to closed development; fewer modern algorithms.	Aggressive improvements: O(1) scheduler (earlier), CFS, modern MM, NUMA support, etc.

Despite these differences, however, Linux remains an operating system with a strong Unix heritage.

Linux Version naming

The first value is the major release, the second is the minor release, and the third is the revision. An optional fourth value is the stable version.

The minor release also determines whether the kernel is a stable or development kernel; an even number is stable, whereas an odd number is development.

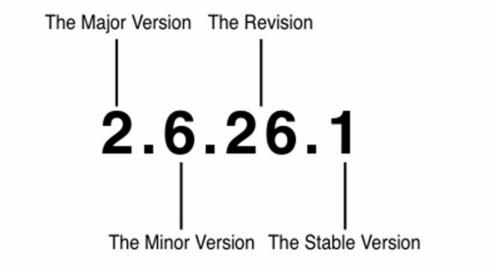


Figure 1.2 Kernel version naming convention.

For example, the kernel version 2.6.30.1 designates a stable kernel. This kernel has a major version of two, a minor version of six, a revision of 30, and a stable version of one.

The first two values describe the "kernel series"—in this case, the 2.6 kernel series.

Operating System Services

An operating system provides an environment for the execution of programs.

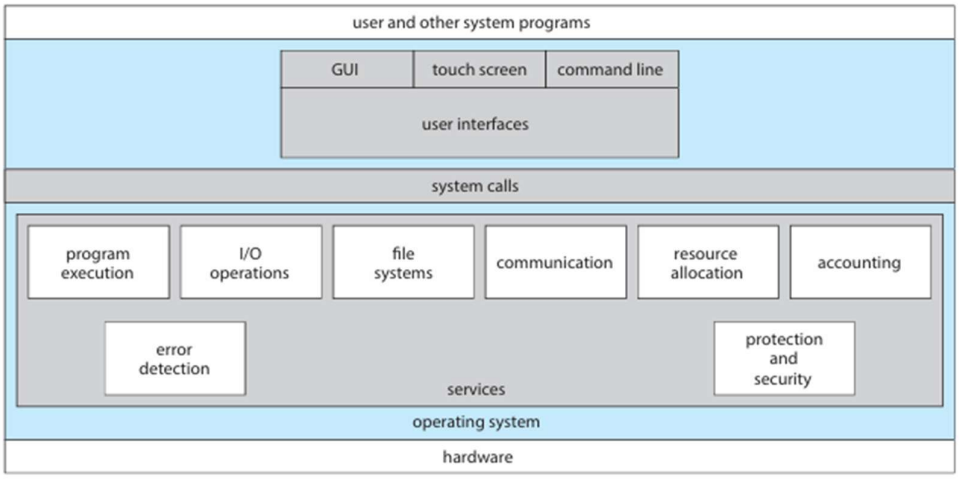


Figure 2.1 A view of operating system services.

- **User interface** - Almost all operating systems have a user interface (UI). This interface can take several forms.

One of the examples is a **graphical user interface (GUI)** Here, the interface is a window system with a mouse that serves as a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.

Mobile systems such as phones and tablets provide a **touch-screen interface**, enabling users to slide their fingers across the screen or press buttons on the screen to select choices

Another option is a **command-line interface(CLI)**,which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options). Some systems provide two or all three of these variations.

- **Program execution** – The system must be able to load a program into memory and to run that program.
- **I/O operations** - A running program may require I/O, which may involve a file or an I/O device. For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.
- **File-system manipulation**- The OS manages the file system by organizing the files , tracking where they are stored , handling naming , access and permissions.
- **Communications** - There are many circumstances in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a network.

  Communications may be implemented via **shared memory**, in which two or more processes read and write to a shared section of memory, or **message passing**, in which packets of information in predefined format are moved between processes by the operating system.

- **Error detection** – The operating system needs to be detecting and correcting errors constantly. Errors may occur in the CPU and memory hardware(such as a memory error or a power failure), In I/O devices(such as a parity error on disk, a connection failure on a network, or lack of paper in the printer), and in the user program (such as an arithmetic overflow or an attempt to access an illegal memory location).
- **Resource allocation -** When there are multiple processes running at the same time, resources must be allocated to each of them.
- **Logging -** We want to keep track of which programs use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.
- **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information. When several separate processes execute concurrently, it should not be possible for one process to interfere with the others or with the operating system itself. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important.

Building/installing Operating systems

If you are generating (or building) an operating system from scratch, you must follow these steps:

1. Write the operating system source code (or obtain previously written source code).

2. Configure the operating system for the system on which it will run.

3. Compile the operating system.

4. Install the operating system.

5. Boot the computer and its new operating system.

Configuring the system involves specifying which features will be included, and this varies by operating system.

How to build a Linux system from scratch, where it is typically necessary to perform the following steps:

1. Download the Linux source code from http://www.kernel.org.

2. Configure the kernel using the "make menuconfig" command. This step generates the .config configuration file.

3. Compile the main kernel using the "make" command. The make command compiles the kernel based on the configuration parameters identified in the .config file, producing the file vmlinuz, which is the kernel image.

4. Compile the kernel modules using the "make modules" command. Just as with compiling the kernel, module compilation depends on the configuration parameters specified in the .config file.

5. Use the command "make modules install" to install the kernel modules into vmlinuz.

6. Install the new kernel on the system by entering the "make install" command.

When the system reboots, it will begin running this new operating system.

Alternatively, it is possible to modify an existing system by installing a Linux virtual machine. This will allow the host operating system (such as Windows or macOS) to run Linux. This option simply requires downloading the appliance and installing it using virtualization software such as VirtualBox or VMware.

The steps are as follows:

1. Downloaded the Ubuntu ISO image from https://www.ubuntu.com/

2. Instructed the virtual machine software VirtualBox to use the ISO as the bootable medium and booted the virtual machine .

3. Answered the installation questions and then installed and booted the operating system as a virtual machine.

Process

A process can be defined as a program in execution or simply as a running program.

When a program is stored on disk, it is just passive code. It becomes a process only when the operating system loads it into memory, allocates CPU time, and maintains all information needed to run it.

A process contains the program's code, data, stack, heap, CPU register values, open files, and its own private address space.

Each process executes independently and is tracked by the OS using a unique identifier called the PID.

Traditional UNIX systems identify the process init as the root of all child processes.

Init is assigned a pid of 1, and is the first process created when the system is booted. In modern Linux this first process is known as systemd.

Process Creation

In UNIX-like systems, process creation usually begins with the fork() system call. When a process calls fork(), the OS creates a new process called the child, which starts as a near-exact copy of the parent.

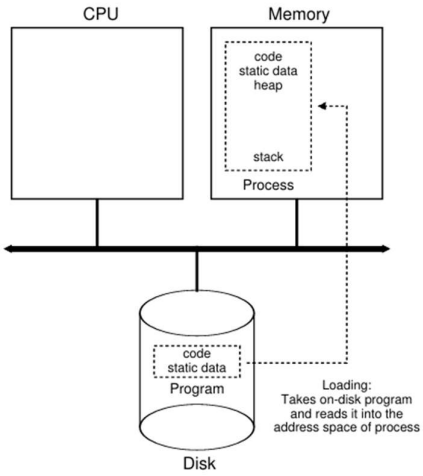


Figure 4.1: **Loading: From Program To Process**

To avoid wasting time and memory, this copying is optimized using techniques like copy-on-write (COW).So, both parent and child access a shared memory space at first and separate page will be created for those who write for themselves.

- You could visualize the currently running processes by typing the command pstree or ps or top or htop commands along with the user preferred options in the terminal window.

.

What is fork()?

- fork() is a system call used to create a new process; included in <unistd.h> library.
- The new process is called the child, and the existing process is the parent.
- Both parent and child execute the same code
- Execution continues from the next statement after fork()
- fork() returns the values like:
  0 → child process
  > 0 → parent (child's PID)
  -1 → error
- getpid() – A system call which returns PID of current process
- getppid()- A system call which returns PID of parent process
- After fork(), the child often calls exec() to replace its memory with a new program.

Example: C program to demonstrate creation of new process and displaying their pids.

```
#include <stdio.h>
#include <unistd.h>

int main() {
   pid_t  process_id;          //pid_t is a data type and process_id is the variable name.

   process_id = fork();

   if (process_id == 0)         // Child process is created

{
       printf("Child 's pid is : %d\n", getpid());
      sleep(20);                                    // keep child alive for some time(optional)

   }
else
{
      printf("Parent id is:%d", getppid());
      sleep(20);                                              // keep parent alive
   }

   return 0;
}
```

Output
Child id: 1034
Parent id : 1032

Or
Parent id : 1032
Child id: 1034

Example 2

```
#include <stdio.h>
#include <unistd.h>

int main()
 {
   printf("Hello\n");
   fork();
   printf("World\n");
   return 0;
}
```

Output
Hello
World
World

Explanation
printf("Hello\n") → executed once
fork() → creates 2 processes
Both parent and child execute printf("World\n")

Example 3

```
#include <stdio.h>
#include <unistd.h>

int main() {
   printf("A\n");   // one printf above fork()

   fork();         // first fork
   fork();         // second fork

   printf("B\n");   // first printf after forks
   printf("C\n");   // second printf after forks
```

```
    return 0;
}
```

Output

```
A                   //print only once
B
C
B
C
B
C
B
C
```

We can't predict which process executes first. As per modern operating system configurations, the OS scheduler will decide which process run first. We can control it by using wait() system calls.

Generally, we can say that if 'n' is the total number of fork() calls,

Total no of processes = 2^n

*It's a general formula and works well only if all forks are executed unconditionally.

wait() and waitpid() System Calls

Why wait() is needed?

The parent always monitors the execution status of its child processes and update the process table. So if the parent process finished its execution, then child process does not have a parent to report its exit status. These types of processes are known as Zombie process.

Denoted by 'Z' as their state.

Eg: Write a C program and create new process (without wait()) and execute it in one terminal see the status of child process by executing htop in another terminal

In order to prevent Zombie processes, the parent may wait for the child to finish its execution using wait() or wait pid () system calls. The parent resumes its operation once the child is terminated.

Wait () - Parent waits for any child to terminate

Syntax: wait(NULL)

Collects exit status

Frees child's process table entry

waitpid() system call - Wait for specific child

Syntax : waitpid(pid, &status, options)

Example :

```
#include <stdio.h>
#include <unistd.h>
#include<sys/wait.h>

int main() {
   pid_t  process_id;          //pid_t is a data type and process_id is the variable name.

   process_id = fork();

   if (process_id == 0)         // Child process is created

{
       printf("Child ‘s pid is : %d\n", getpid());


   }
else
{
     wait(NULL);                                          // keep parent in a waiting state
      printf("Parent id is:%d", getppid());

   }

   return 0;
}
```

Output

Child id : 399 //Child executes first before parent process

Parents id: 1024

Process states

A process does not stay in one condition throughout its lifetime. As it runs, it moves through several states.

When it is first created, it enters the **new** state. Once ready to run but waiting for CPU time, it is in the **ready** state. When the CPU begins executing its instructions, it enters

the **running** state. If the process needs to wait for an external event, such as disk I/O or user input, it becomes **blocked**. When the event completes, it returns to the ready state. After finishing execution or being terminated, it moves into the **terminated** state. This cycle ensures smooth coordination of many processes using shared CPU resources.

The main states are listed below:

- **New** state : When it is first created, it enters the **new** state
- **Running**: In the running state, a process is running on a processor. This means it is executing instructions.
- **Ready**: In the ready state, a process is ready to run but for some reason the OS has chosen not to run it at this given moment.
- **Blocked**: In the blocked state, a process has performed some kind of operation that makes it not ready to run until some other event takes place.
- **Terminated** state - After finishing execution or being terminated, it moves into the **terminated** state

  A common example: when a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.

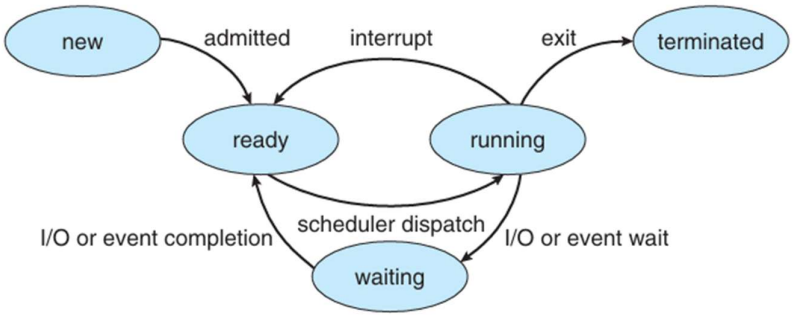


Figure 3.2 Diagram of process state.

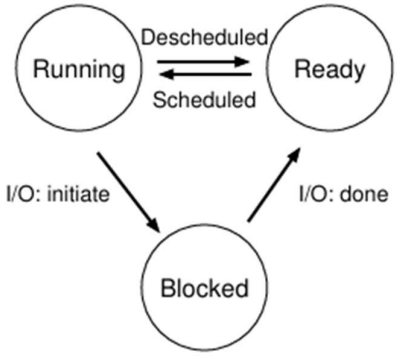
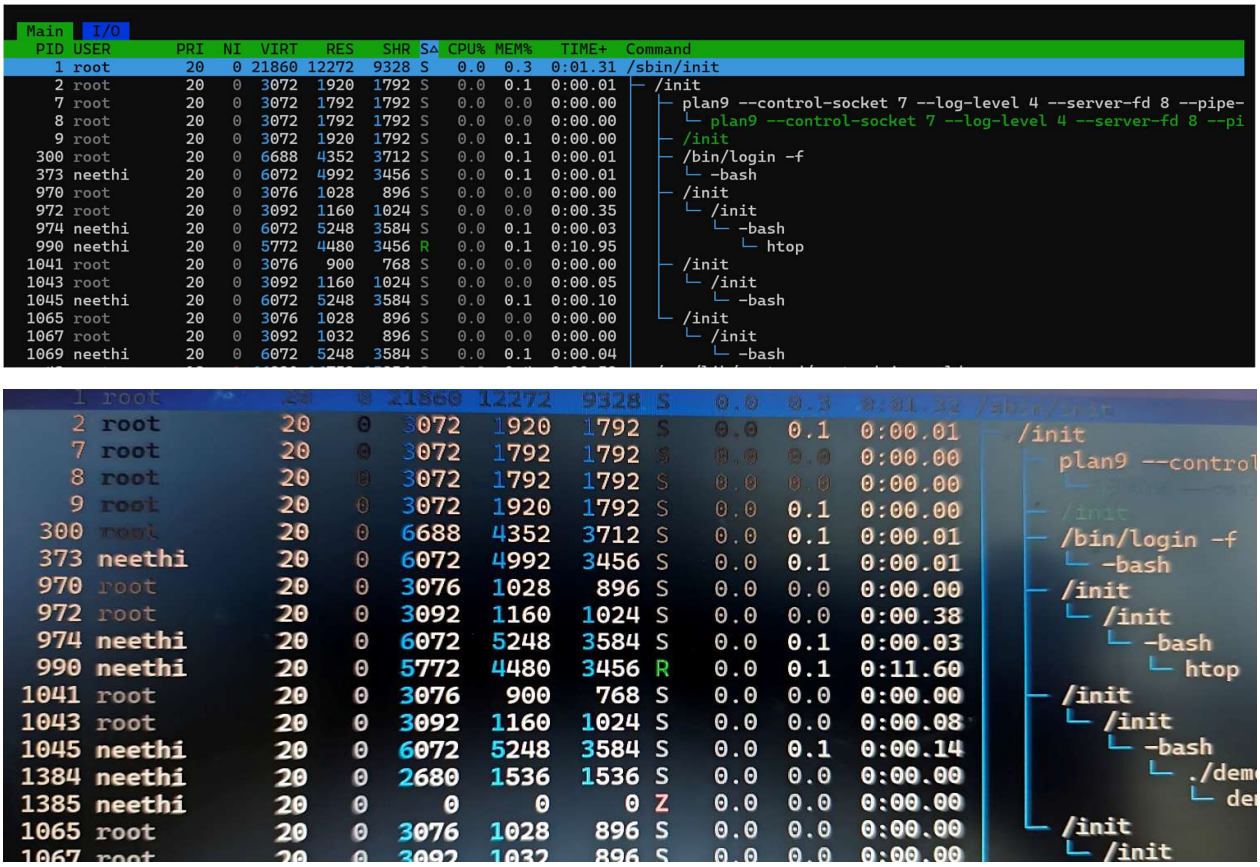


Figure 4.2: **Process: State Transitions**

Demo: Type htop in terminal and see the processes which are currently running along with their process state.
S denotes sleeping state
R denotes Running
Z denotes Zombie process
T denotes terminated process



Process Data Structures

To manage each process efficiently, the operating system maintains an internal data structure known as the **Process Control Block (PCB)**.

The PCB contains critical information about the process, including its PID, current state, program counter, CPU register contents, memory boundaries, open file descriptors, scheduling information, and parent/child relations.

Whenever the OS switches between processes(context switching), it saves and restores PCB contents. This makes the PCB the “identity and memory” of a process within the operating system.

Without this structure, the OS would not be able to pause, resume, or track processes properly.

It includes

- CPU registers : The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
- The program counter : It has the address of next instruction to be fetched.
- CPU-scheduling information - This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

Figure 3.3 Process control block (PCB).

- Memory-management information - This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.
- Accounting information - This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- I/O status information. This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

Process API

The process API represents the set of system calls provided by the OS to create, control, and terminate processes.

Since the OS is responsible for process creation, scheduling, memory allocation, and termination, user programs cannot perform these operations directly. Instead, they request these services through the process API. These operations are usually implemented as **system calls**, which transfer control into the kernel.

- The most important among them is **fork()**, which creates a new process.
- The **exec()** family of calls loads a new program into the process's address space. The exec() family of functions completely replaces the calling process's memory with

a new program loaded from disk. The process keeps its PID and other kernel-level identity but loses its old code and data. After a successful exec(), the process begins executing the new program from its entry point. The combination of fork() followed by exec() is the standard method to run a new program in UNIX: first create a process, then transform it into the program you want to run.

- The **wait()** call allows a parent process to pause until its child process finishes execution.

  Typically, a parent process needs to check the status of its children, particularly to ensure they have finished or exited properly. The wait() system call allows a parent to pause its own execution until one of its children terminates. This coordination prevents so-called "zombie processes," which are processes that have finished execution but still occupy kernel data structures because no parent has collected their exit status. The wait mechanism ensures clean and orderly process termination.

  The moment the parent calls wait(), the parent process **stops executing temporarily**. It enters a *blocked* state, meaning it pauses until one of its child processes finishes. The operating system now monitors the child, and the parent stays inactive while waiting. As soon as the child completes its execution and calls exit(), the kernel stores the child's exit status and marks the child as "terminated but not yet reaped."
  At this point, the kernel immediately wakes up the parent process, because the event the parent was waiting for has now happened.

  The parent then resumes execution from the point after wait() and collects the child's exit information. This final step of collecting the child's exit status is called **reaping**, which completely removes the child's entry from the OS process table. Thus, the presence of wait() ensures that no zombie processes remain, resources are properly released, and the parent-child relationship ends cleanly.

- The **exit()** call enables a process to end its execution voluntarily and return a status. When a process calls exit(), it informs the OS that it is finished and returns an exit status to the parent. The kernel then frees the resources allocated to the process, closes its files, and updates its status so the parent can retrieve it

- The **kill ()** allow processes to send signals to one another. These APIs form the fundamental tools through which applications interact with the OS to manage their execution flow.

  The kill() system call, despite its name, does not always terminate processes; it is actually a general-purpose mechanism for sending signals. Signals can be used to stop, continue, interrupt, or request termination of a process.

- Other useful API calls include **getpid(),** which returns the process's unique identifier, and **getppid(),** which retrieves the parent's PID. These are essential for processes to understand their relationship within the system.

Sharing the Processor Among Processes

Because the CPU can run only one instruction at a time, the operating system must create the illusion that multiple processes run simultaneously. This is achieved through a mechanism called **time sharing**.

The scheduler divides CPU time into small intervals, called time slices or quanta, and assigns these slices to processes in turn. When a process's time slice expires, a hardware timer interrupts the CPU and triggers a context switch, where the OS saves the current process's state and loads the next one's state. By switching rapidly among processes, the system gives users the feeling that many programs are running at the same time.

User Mode and Kernel Mode

Modern operating systems operate with two protection levels: **user mode** and **kernel mode**.

In user mode, ordinary applications run with limited privileges. They cannot directly access hardware, modify system memory, or perform sensitive operations. When a program needs such access, it must call a system call. The CPU then switches into kernel mode, where the operating system runs with full privileges.

Kernel mode (**privilege mode**) allows access to hardware devices, memory management structures, and other protected resources. Once the kernel completes the requested operation, the CPU switches back to user mode. This separation ensures stability and security by preventing user programs from damaging the system.

Context Switching

A context switch is the process of saving the state of a currently running process or thread so that it can be resumed later, and restoring the state of another process or thread to start or continue its execution.

It occurs when the CPU switches from executing one process to another.

This is fundamental for multitasking in operating systems.

The CPU can switch between multiple processes, giving the illusion that they run simultaneously, even though the CPU executes only one at a time.

Why context switching is needed?

Suppose we have a single CPU and that can execute only one process at a time, but multiple processes appear to run concurrently.

When a process cannot continue (e.g., it is waiting for I/O) or a higher-priority process is ready or time slice expires for that process, then OS performs a context switch to allow the CPU to execute another process.

Steps involved in a context switch

1. Save the context of the currently running process:

ie, Store CPU registers (PC, SP, general-purpose registers) and program status word.

Store memory-management information, if needed.

Update process control blocks (PCB)

2. The current process state changes from Running → Ready or Waiting.
3. Select a new process to run:
4. Scheduler picks the next process from the ready queue.
5. Restore the context of the new process:
6. Load CPU registers, program counter, stack pointer, and memory info.
7. Start/resume execution of the new process.

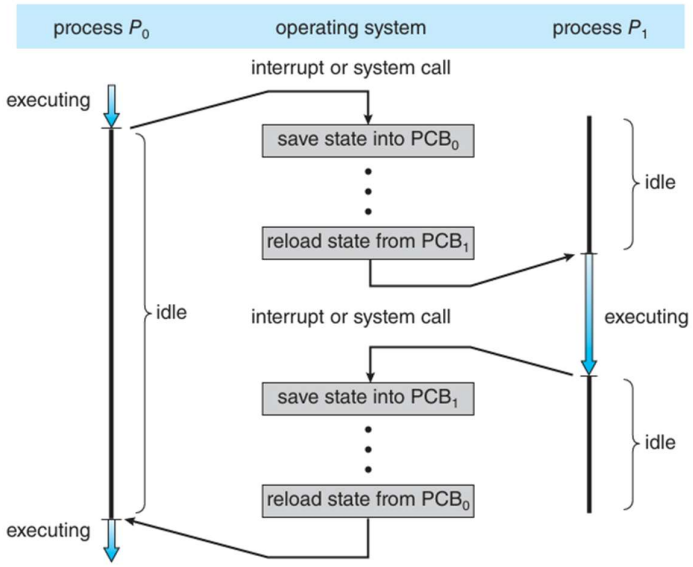


Figure 3.6 Diagram showing context switch from process to process.

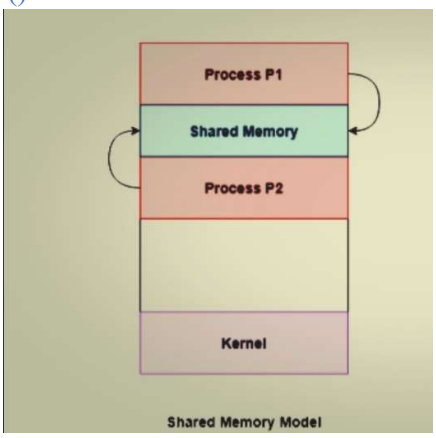
Inter processes Communication / IPC

IPC is a mechanism that allows processes to communicate and share data with each other. It is required because processes have separate memory spaces in most operating systems.

Types of IPC

1.Shared Memory

- Processes share a region of memory.
- Fastest form of IPC because no kernel intervention is required after setup.
- Requires synchronization mechanisms (like semaphores or mutex) to avoid race conditions.
- In C, use shmget() to create/get a shared memory segment.
- Use shmat() to attach it to process memory.
- Access the memory like a normal pointer.
- shmdt() to detach, shmctl() to delete.

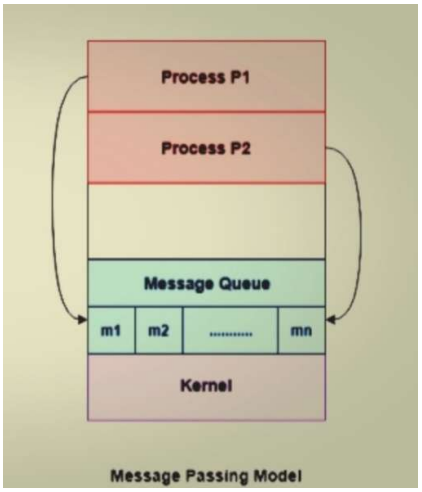


Example: Two processes updating a shared array.

2.Message Passing

- Processes send and receive messages through the operating system.
- No shared memory needed.
- Two forms: Synchronous (blocking): Sender/receiver waits until message is delivered. Asynchronous (non-blocking): Sender continues without waiting.

Example: Pipes, message queues.



3.Pipes

- Unidirectional channel between related processes (parent-child).
- Anonymous pipes: used for processes in the same program.
- Named pipes (FIFOs): used for unrelated processes.
- In C, use pipe(int fd[2]) to create a pipe where (fd[0] for read, fd[1] for write).
- Child or Parent can write or read through the pipe.

4.Message Queues

- Queue maintained by OS to store messages until the receiving process retrieves them.
- Allows communication between unrelated processes.
- In C, u msgget() to create/get a message queue.
- Use msgsnd() to send messages.
- Use msgrcv() to receive messages.
- msgctl() to remove the queue.

5.Semaphores / Signals

- Semaphores: Used for synchronization, not direct data transfer.
- Signals: Notify a process that a particular event has occurred.
- In C, use semget() to create/get a semaphore set.
- Use semop() to perform P (wait) or V (signal) operations.
- semctl() to control/remove semaphores.

6.Sockets

- Communication between processes over network or same machine.
- Can be stream-oriented (TCP) or datagram-oriented (UDP).
- Use socket() to create a socket.
- bind(), listen(), accept() for server side.
- connect() for client side.

- Use send() and recv() to transfer data.

Independent Process and Cooperating Process

Independent Process

- A process that does not affect or is not affected by other processes.
- No data sharing or communication with other processes.

Example: A simple program calculating factorial.

Cooperating Process

- A process that can affect or be affected by other processes.
- May share data or coordinate execution.

Example: Multiple processes updating a shared bank account balance.

Case Study : Linux Kernel Process Management

Process Concept in Linux

- A **process** is a running instance of a program.
- In Linux, processes are represented by **task_struct** (a kernel data structure).

  The kernel stores the list of processes in a circular doubly linked list called the task list.

  The process control block in the Linux operating system is represented by the C structure **task_ struct**, which is found in the include file in the kernel source-code directory.

  This structure contains all the necessary information for representing a process, including the state of the process, scheduling and memory-management information, list of open files, and pointers to the process's parent and a list of its children and siblings. Each element in the task list is a process descriptor of the type struct task_struct, which is defined in <linux/sched.h>

  The process descriptor contains all the information about a specific process. The task_struct is a relatively large data structure, at around 1.7 kilobytes on a 32-bit machine. The process descriptor contains the data that describes the executing program—open files, the process's address space, pending signals, the process's state, and much more,

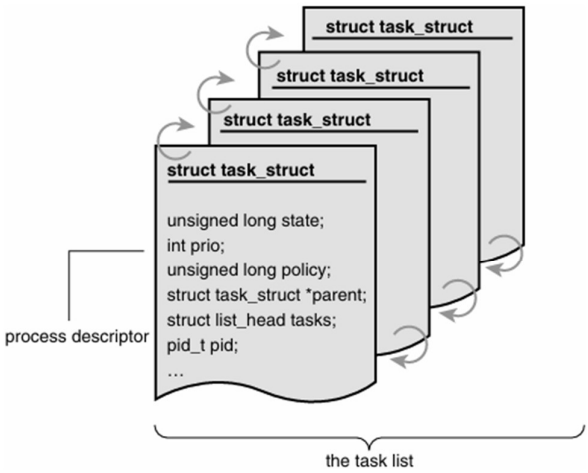


Figure 3.1 The process descriptor and task list.

- Every process has:
  - **PID (Process ID)** - The system identifies processes by a unique process identification value or PID.
  - **Parent PID (PPID)**
  - **State** (running, sleeping, stopped, zombie)
  - **CPU context** (registers, program counter)
  - **Memory space** (code, data, stack)
  - **Open files** and other resources

2. Process States - Each process on the system is in exactly one of five different states. This value is represented by one of five flags

- **TASK_RUNNING** – process can run or is running on CPU.
- **TASK_INTERRUPTIBLE** – sleeping, can be awakened by signals.
- **TASK_UNINTERRUPTIBLE** – sleeping, cannot be awakened by signals.
- **TASK_STOPPED** – stopped by signal.
- **TASK_ZOMBIE** – terminated but parent hasn't read exit status yet.

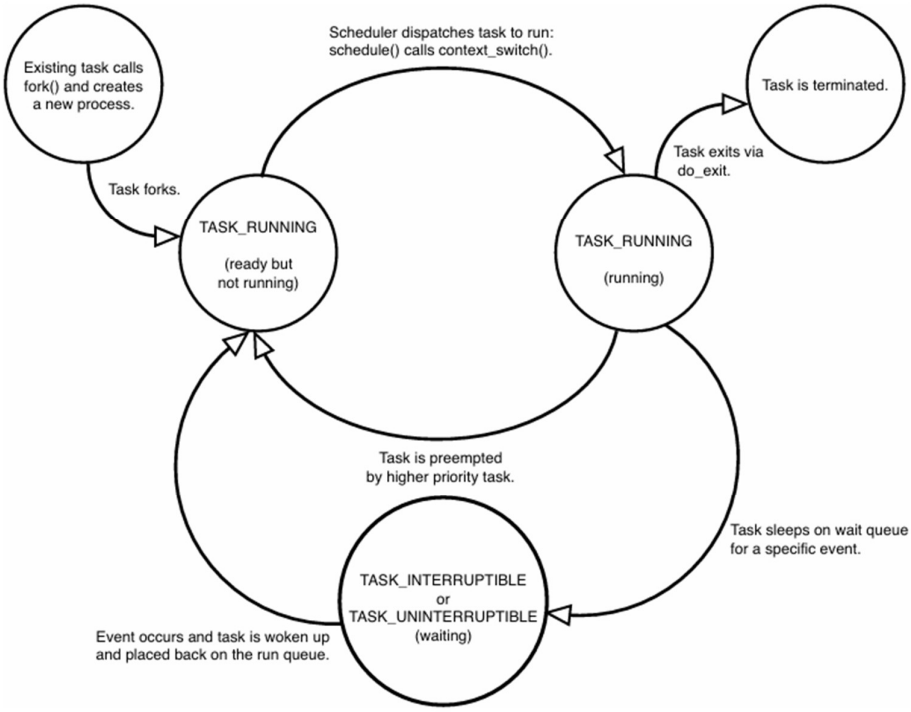


Figure 3.3 Flow chart of process states.

3. Process Creation

- Linux uses **fork()** system call to create a new process.
- Linux implements fork() via the clone() system call internally.
- **fork()** duplicates the parent process:
  - Child gets a copy of parent's memory space.
  - **Copy-on-write (COW)** avoids unnecessary copying – its an optimization technique in process management where parent and child process initially share same memory pages after fork(), but actual copying of those pages occurs only when either of them tries to modify them. So, until a write occurs, all shared pages remain read only and not duplicated, saving time and memory.
- Kernel creates a new task_struct for the child.

init process (PID 1) is the ancestor of all processes in Linux. All processes are descendants of the init process, whose PID is one.

The kernel starts init in the last step of the boot process. The init process, in turn, reads the system initscripts and executes more programs, eventually completing the boot process.

- **vfork()**: Similar to fork but designed for efficiency when child immediately calls exec().

- **clone()**: More flexible than fork; allows sharing of memory, file descriptors, etc. (used in threads).

4. Process Scheduling

- Linux uses a **preemptive scheduler**.
- Each process has:
    - **Static priority** and **dynamic priority**
    - **Time slices**

It uses **Completely Fair Scheduler (CFS)** (Linux 2.6+):

- Uses a **red-black tree** to track runnable processes.
- Ensures fair CPU distribution based on **virtual runtime**.

5. Context Switching

- Switch from one process to another:
    1. Save current process's CPU registers.
    2. Update task_struct with state.
    3. Load next process's CPU registers.
- Context switch happens:
    - On timer interrupt
    - When a process blocks for I/O
    - When a higher priority process becomes runnable

6. Kernel vs User Processes

- Kernel processes (like kswapd) run in kernel mode.
- User processes run in user mode but can make system calls to access kernel services.
- Each process has **two stacks**:
    - **Kernel stack**
    - **User stack**

7. Inter-process Relationships

- Processes form a **tree structure**:
    - Each process has a parent.
    - init is root.
- **Orphan processes**: Parent terminates, child adopted by init.

- **Zombie processes**: Child terminates, parent hasn't called wait().

8. Process Termination

- exit() system call is used.
- Kernel:
    - Releases resources (memory, file descriptors).
    - Notifies parent via **SIGCHLD**.
- Parent calls wait() to get child's exit status.

9. Threads (Lightweight Processes)

- Threads are created with **clone()** system call.
- Share some resources (memory, file descriptors) but have own **task_struct** and **kernel stack**.
- Linux sees threads as **processes that share resources**.

System Boot

The process of starting a computer by loading the kernel is known as booting the system.

At first ,

- The system performs **POST (Power-On Self-Test) -** Performed by motherboard firmware (BIOS/UEFI) which resides on a non-volatile memory chip (EEPROM/Flash ROM) on the motherboard.

  It Checks hardware: CPU, RAM, disk, keyboard, etc.
- If successful → firmware loads the bootloader.

When the computer is first powered on, a small boot loader located in non volatile firmware known as BIOS is run. This BIOS searches for a bootable device which follows the boot order you set (Eg – SSD-> HDD->USB->Network etc).

Then the BIOS loads the MBR (Master Boot Record) from first sector of the boot able device. This contains a small program known as bootstrap program / bootstrap loader. Its job is to load the rest of the OS kernel into memory. The kernel initialises essential components like CPU, memory management, interrupts and essential drivers at first. Rest of the OS will be loaded when needed.

Many recent computer systems have replaced the BIOS-based boot process with UEFI (Unified Extensible Firmware Interface). UEFI has several advantages over BIOS, including better support for 64-bit systems and larger disks.

Perhaps the greatest advantage is that UEFI is a single, complete boot manager and therefore is faster than the multistage BIOS boot process.

In addition to loading the file containing the kernel program into memory, it also runs diagnostics to determine the state of the machine

The bootstrap can also initialize all aspects of the system, from CPU registers to device controllers and the contents of main memory. Sooner or later, it starts the operating system and mounts the root file system. It is only at this point is the system said to be running.

On most systems, the boot process proceeds as follows:

1. A small piece of code known as the bootstrap program or boot loader locates the kernel.

2. The kernel is loaded into memory and started.

3. The kernel initializes hardware.

4. The root file system is mounted.

In Linux

In Linux, GRUB is an open-source bootstrap program for Linux and UNIX systems. Boot parameters for the system are set in a GRUB configuration file, which is loaded at startup. GRUB is flexible and allows changes to be made at boot time, including modifying kernel parameters and even selecting among different kernels that can be booted.

To save space as well as decrease boot time, the Linux kernel image is a compressed file that is extracted after it is loaded into memory. During the boot process, the boot loader typically creates a temporary RAM file system, known as initramfs. This file system contains necessary drivers and kernel modules that must be installed to support the real root file system (which is not in main memory).

Once the kernel has started and the necessary drivers are installed, the kernel switches the root file system from the temporary RAM location to the appropriate root file system location. Finally, Linux creates the systemd process, the initial process in the system, and then starts other services (for example, a web server and/or database). Ultimately, the system will present the user with a login prompt.

In Android Systems

The boot process for mobile systems is slightly different from that for traditional PCs. For example, although its kernel is Linux-based, Android does not use GRUB and instead leaves it up to vendors to provide boot loaders.

The most common Android boot loader is LK (for “little kernel”). Android systems use the same compressed kernel image as Linux, as well as an initial RAM file system. However, whereas Linux discards the initramfs once all necessary drivers have been loaded, Android

maintains initramfs as the root file system for the device. Once the kernel has been loaded and the root file system mounted, Android starts the init process and creates a number of services before displaying the home screen.

Finally, boot loaders for most operating systems—including Windows, Linux, and macOS, as well as both iOS and Android—provide booting into recovery mode or single-user mode for diagnosing hardware issues, fixing corrupt file systems, and even reinstalling the operating system.

In Windows

- After POST, it loads the Windows bootloader which can be either **bootmgfw.efi** (UEFI) or **bootmgr** (BIOS).

The Windows Boot Manager reads **Boot Configuration Data (BCD)** , decides which Windows version to start (useful in dual-boot) and launches the Windows OS loader: **winload.exe**. This loader loads windows kernel (ntoskml.exe) ,hardware abstraction layer and essential drivers to start the system. The very first user mode process - SMSS.exe (session manager) started directly by kernel.

Threads

A thread is a basic unit of CPU utilization within the process ; It comprises a

- thread ID,
- program counter (PC),
- register set,
- stack.

It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

- In Linux, a **thread is essentially a lightweight process (LWP – Light Weight Process)**.
- Threads share the **same address space, open files, and signal handlers** but have **their own execution context**:
    - Program counter
    - Stack
    - CPU registers

A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.

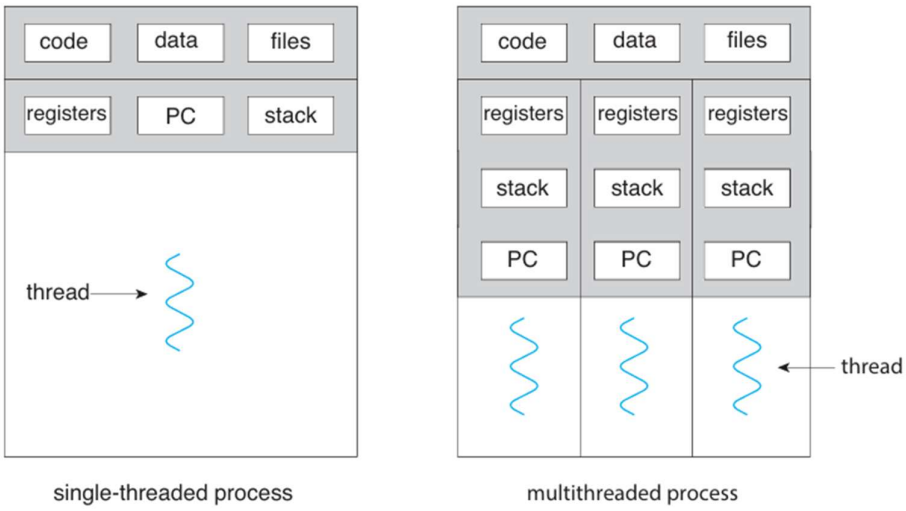


Figure 4.1 Single-threaded and multithreaded processes.

Example: A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

Pthreads refers to the POSIX standard (IEEE 1003.1c) defining an API for thread creation and synchronization.

How to implement threads in C?

- Threads are included in <pthread.h> library.
- Thread identifier can be declared as

```
Pthread_t thread1, thread2;
```

- The thread function can be as

```
void*  function_name (void* arg)
  {
     ------------------
     ------------------
  }
 return NULL;
```

- Creation of threads

```
Pthread_create (&thread1,NULL,myThreadFunction,NULL);
                                          //NULL as default parameter.
```

- Inorder to control the order of execution, pthread_join is used, So the main program will wait for threads to finish .

Pthread_join(thread1,NULL);

//To ensure that the main program waits until the thread finishes.

- To exit a thread

  Pthread_exit(NULL);

Sample program to demonstrate the working of threads

```
#include <pthread.h>
#include <stdio.h>

int a = 20;                    //initialized as global variables.
int b = 5;

// Thread function to add
void* add(void* arg)
{
   printf("Addition: %d + %d = %d\n", a, b, a + b);
   return NULL;
}

// Thread function to subtract
void* subtract(void* arg)
 {
   printf("Subtraction: %d - %d = %d\n", a, b, a - b);
   return NULL;
}

// Thread function to multiply
void* multiply(void* arg)
{
   printf("Multiplication: %d * %d = %d\n", a, b, a * b);
   return NULL;
}

// Thread function to divide
void* divide(void* arg)
{
   if(b != 0)
      printf("Division: %d / %d = %d\n", a, b, a / b);
   else
      printf("Division by zero error!\n");
   return NULL;
}
```

```
int main()
 {
   pthread_t t1, t2, t3, t4;

   // Create threads
   pthread_create(&t1, NULL, add, NULL);
   pthread_create(&t2, NULL, subtract, NULL);
   pthread_create(&t3, NULL, multiply, NULL);
   pthread_create(&t4, NULL, divide, NULL);

   // Wait for threads to finish
   pthread_join(t1, NULL);
   pthread_join(t2, NULL);
   pthread_join(t3, NULL);
   pthread_join(t4, NULL);

   printf("All threads finished execution.\n");
   return 0;
}
```

Output

Subtraction: 20 - 5 = 15

Addition: 20 + 5 = 25

Division: 20 / 5 = 4

Multiplication: 20 * 5 = 100

All threads finished execution.

Multithreaded architecture

Multithreading allows programs to do multiple activities at once, each handled by a separate thread to ensure concurrency.

Benefits are

- **Faster execution** (parallel tasks)
- **Better resource utilization**
- **Improved program structure** (divide tasks into separate threads)

Example: In a web server:

- One thread handles client requests.
- Another thread writes logs and service the request.
- Another thread interacts with the database and listen for additional client requests.

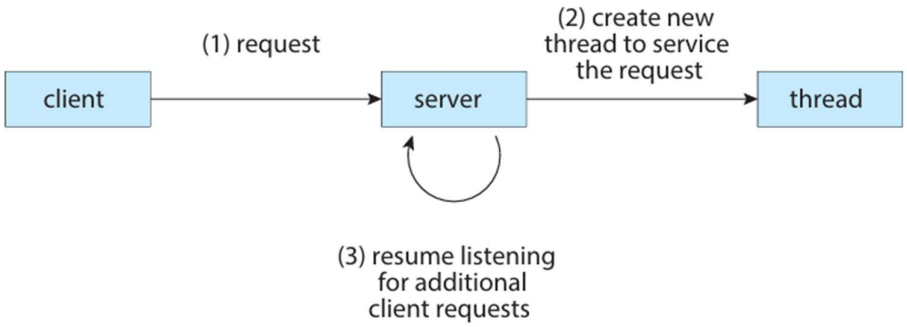


Figure 4.2 Multithreaded server architecture.

Benefits of multithreaded programming / Why threads?

The benefits of using multithreaded programming can be broken down into four major categories:

1. **Responsiveness** - Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. This quality is especially useful in designing user interfaces.

For instance, consider what happens when a user clicks a button that results in the performance of a time-consuming operation. A single-threaded application would be unresponsive to the user until the operation had been completed. In contrast, if the time-consuming operation is performed in a separate, asynchronous thread, the application remains responsive to the user.

2. **Resource sharing** -Processes can share resources only through techniques such as shared memory and message passing. Such techniques must be explicitly arranged by the programmer. However, threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

3. **Economy** - Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads. In general thread creation consumes less time and memory than process creation. Additionally, context switching is typically faster between threads than between processes.

4**. Scalability** - The benefits of multithreading can be even greater in a multiprocessor architecture, where threads may be running in parallel on different processing cores.

A single-threaded process can run on only one processor, regardless how many are available.

Multithreaded Models

1.Many- to- One model

The many-to-one model maps many user-level threads to one kernel thread.

Thread management is done by the thread library in user space , so it is efficient.

However, the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.

Eg - Green threads—a thread library available for Solaris systems and adopted in early versions of Java—used the many-to one model.

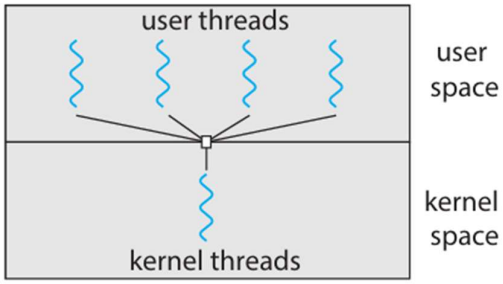


Figure 4.7 Many-to-one model.

2. One-to-One Model

The one-to-one model maps each user thread to a kernel thread.

It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.

It also allows multiple threads to run in parallel on multiprocessors.

The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system.

Eg - Linux, along with the family of Windows operating systems, implement the one-to-one model.

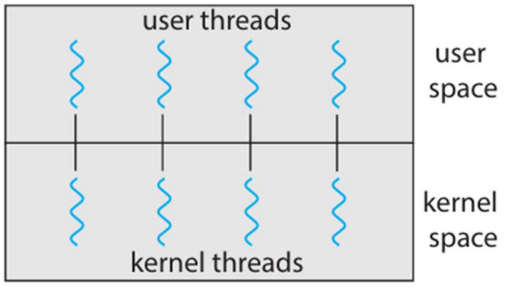


Figure 4.8 One-to-one model.

3. Many-to-Many Model

The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.

The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a system with eight processing cores than a system with four cores).

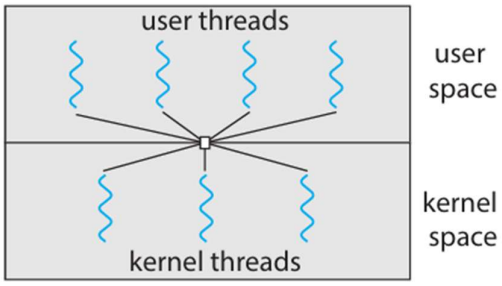


Figure 4.9 Many-to-many model.

Difference between processes and threads

Processes	Threads
It's a program in execution and is a heavy unit of execution.	A lightweight unit of execution within a process
Has a separate address space	Shares the same address space of the process
Does not share memory with other processes	Shares memory, files, and resources
Heavyweight; creation is slow	Lightweight; creation is fast

Context switching is costly Communication via IPC (pipes, sockets, shared memory)	Context switching is less costly Communication via shared variables
Failure of one process does not affect others	Failure of one thread can terminate the entire process
Example: Running two different programs	Example: Multiple threads in a browser

Case Study: Linux implementation of Threads.

Creating Threads

- **In Linux, a thread is essentially a lightweight process (LWP – Light Weight Process).**
- Threads share the same address space, open files, and signal handlers but have their own execution context:
    - Program counter
    - Stack
    - CPU registers

Kernel-Level Threads (KLT)

- The kernel schedules threads **independently**, just like processes.
- Created **directly by the kernel** using **clone()**.
- Each kernel thread has a **task_struct**, scheduled independently by the kernel.

Example: **Linux system threads** like kthreadd are kernel threads.

User Threads (Pthreads)

- **Pthreads (POSIX threads)** are a **user-level API** for creating threads.
- Pthreads can be implemented in **two ways**:

1. **Many-to-One (User-level threads)** - All user threads mapped to **a single kernel thread**.
2. **One-to-One (Kernel-level threads / Linux default)** - Each pthread maps to a **kernel thread** (via clone() under the hood).

clone() system call - The main way to create a kernel thread in Linux.

- Clone () allows **fine-grained control** over what is shared between the parent and child:
  - **CLONE_VM** → share memory space
  - **CLONE_FS** → share filesystem info
  - **CLONE_FILES** → share file descriptors
  - **CLONE_SIGHAND** → share signal handlers
- Example: clone(thread_fn, stack_top, CLONE_VM | CLONE_FS | CLONE_FILES, arg)

Threads are created the same as normal tasks, with the exception that the clone() system call is passed flags corresponding to the specific resources to be shared:

clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);

The previous code results in behaviour identical to a normal fork(), except that the address space, filesystem resources, file descriptors, and signal handlers are shared.

Thread Libraries

- Linux provides **pthreads (POSIX threads)** as a user-level API.
- **Pthreads** are implemented on top of **kernel threads**.
- Most pthread operations (create, join, mutex, etc.) are translated to **clone() calls and kernel operations**.

Fork() versus Clone()

In Linux, fork() creates a new process with **separate memory space**.

clone() can create a **thread sharing memory and resources**.

Process Scheduling / CPU Scheduling

CPU scheduling is the mechanism of selecting one process from the ready queue and allocating the CPU to it when the CPU becomes idle.

We will make the following assumptions about the processes, sometimes called jobs, that are running in the system:

1. Each job runs for the same amount of time.

2. All jobs arrive at the same time.

3. All jobs only use the CPU (i.e., they perform no I/O)

4. The run-time of each job is known

Scheduling Metrics

- **Turnaround time**

The turnaround time of a job is defined as the time at which the job completes minus the time at which the job arrived in the system. More formally, the turnaround time $T_{turnaround}$ is:

$$T_{turnaround} = T_{completion} - T_{arrival}$$

we have assumed that all jobs arrive at the same time, for n , $T_{turnaround} = T_{completion}$

Includes waiting time + execution time.

ie, Turnaround Time is the total time taken by a process from arrival to completion.

TAT = CT - AT

- **Arrival Time (AT)**

Arrival Time is the time at which a process enters the ready queue and becomes ready for execution.

- **Burst Time (BT)**

Burst Time is the total CPU time required by a process to complete execution.

Does not include waiting or I/O time.

- **Completion Time (CT)**

Completion Time is the time at which a process finishes its execution completely.

- **Waiting Time (WT)**

Waiting Time is the total time a process spends waiting in the ready queue.

WT = TAT - BT

- **Response Time**

Response time is defined as the time from when the job arrives in a system to the first time it is scheduled.

More formally: $T_{response} = T_{firstrun} - T_{arrival}$

Types of CPU Scheduling

1. Non-Preemptive Scheduling

- In non-preemptive scheduling, once the CPU is allocated to a process, the process continues execution until it completes or enters the waiting state.

- The CPU cannot be taken away from the running process.
- Once a process gets the CPU, it runs till completion
- CPU cannot be interrupted
- FCFS, SJF (Non-preemptive) , Priority (Non-preemptive) are the examples

2.Preemptive Scheduling

- In preemptive scheduling, the operating system can interrupt a running process and assign the CPU to another process.

Here, a running process can be stopped when:

Time quantum expires, A higher-priority process arrives , A process with shorter remaining time arrives.

- Examples are Round Robin, SRTF , Preemptive Priority

Scheduling Algorithms

1.) FCFS / FIFO

The most basic algorithm a scheduler can implement is known as First In, First Out (FIFO) scheduling or sometimes First Come, First Served (FCFS).

Imagine three jobs arrive in the system, A, B, and C, at roughly the same time ($T_{arrival} = 0$). Because FIFO has to put some job first, let's assume that while they all arrived simultaneously, A arrived just a hair before B which arrived just a hair before C. Assume also that each job runs for 10 seconds.

What will the average turnaround time be for these jobs?

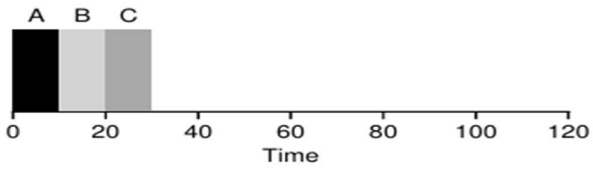


Figure 7.1: **FIFO Simple Example**

FromFigure7.1, you can see that A finished at 10, B at 20, and C at 30. Thus, the average turn around time for the three jobs is simply (10+20+30)/ 3 = 20.

let's again assume three jobs (A, B, and C), but this time A runs for 100 seconds while B and C run for 10 each.

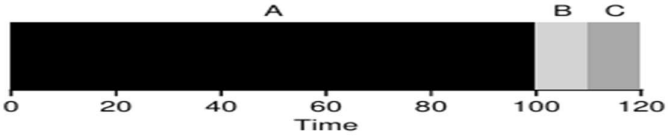


Figure 7.2: **Why FIFO Is Not That Great**

As you can see in Figure 7.2, Job A runs first for the full 100 seconds before B or C even get a chance to run. Thus, the average turnaround time for the system is high: a painful 110 seconds (100+110+120 3 =110).

This problem is generally referred to as the convoy effect, where a number of relatively-short potential consumers of are source get queued behind a heavy weight resource consumer.

2. Shortest job First (SJF)

- it runs the shortest job first, then the next shortest, and so on

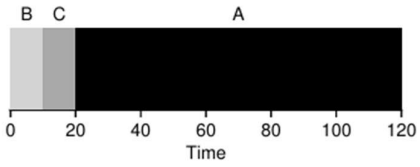


Figure 7.3: **SJF Simple Example**

Simply by running B and C before A, SJF reduces average turnaround from 110 seconds to 50 ie, Avg Trurn around time is (10+20+120)/ 3 = 50.

Another Scenario - This time, assume A arrives at t = 0 and needs to run for 100 seconds, whereas B and C arrive at t = 10 and each need to run for 10 seconds. With pure SJF, we'd get the schedule seen in Figure 7.4.

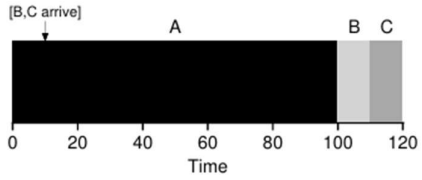


Figure 7.4: **SJF With Late Arrivals From B and C**

As you can see from the figure, even though B and C arrived shortly after A, they still are forced to wait until A has completed, and thus suffer the same convoy problem. Average turnaround time for these three jobs is 103.33 seconds

Ie, (100+(110 −10) + (120−10) / 3).

What can a scheduler do?

3.Shortest Time-to-Completion First (STCF) or SRTF (Shortest Remaining Time First)

Here adds preemption to SJF, known as the Shortest Time-to-Completion First (STCF) or Preemptive Shortest Job First (PSJF) a scheduler.

Any time a new job enters the system, it determines of the remaining jobs and new job, which has the least time left, and then schedules that one. Thus, in our example, STCF would preempt

A and run B and C to completion; only when they are finished would A's remaining time be scheduled. Figure 7.5 shows an example.

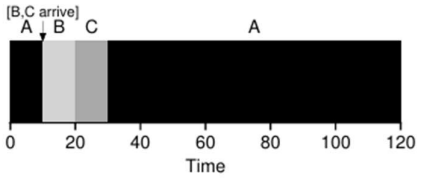


Figure 7.5: **STCF Simple Example**

The result is a much-improved average turnaround time: 50 seconds

Ie, ((120−0)+(20−10)+(30−10)) /3 .

Working of SRTF (Shortest Remaining Time First)

1. When the CPU is free, the process that arrives first is selected and starts execution.

2. During execution, if a new process arrives with a burst time less than the remaining burst time of the currently running process:

The current process is pre-empted - Its context is saved

The new process is allocated the CPU. This will continue until the last process available in the ready queue.

Another Scenario - If three jobs arrive at the same time, for example, the third job has to wait for the previous two jobs to run in their entirety before being scheduled just once. While great for turnaround time, this approach is quite bad for response time and interactivity.

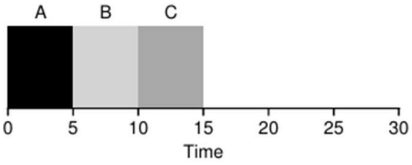


Figure 7.6: **SJF Again (Bad for Response Time)**

To solve this problem, we will introduce a new scheduling algorithm. This approach is classically known as Round-Robin(RR)scheduling.

4.Round-Robin (RR)scheduling.

- instead of running jobs to completion, RR runs a job for a time slice (sometimes called a scheduling quantum) and then switches to the next job in the run queue.

It repeatedly does so until the jobs are finished. For this reason, RR is sometimes called time slicing.

Note that the length of a time slice must be a multiple of the timer-interrupt period; thus if the timer interrupts every 10 milliseconds, the time slice could be 10, 20, or any other multiple of 10 ms.

let's look at an example.
Assume three jobs A, B, and C arrive at the same time in the system, and that they each wish to run for 5 seconds.

An SJF scheduler runs each job to completion before running another (Figure 7.6).
In contrast, RR with a time-slice of 1 second would cycle through the jobs quickly (Figure 7.7).

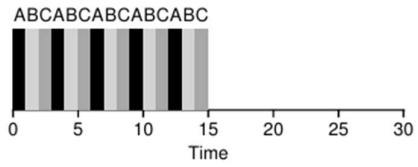


Figure 7.7: **Round Robin (Good for Response Time)**

The average response time of RR is: (0+1+2) / 3 =1.

But for SJF, response time is: (0+5+10)/ 3 =5.

Question

Consider the following set of five processes with their arrival time, burst time, and priority: (Assume: Lower priority number indicates higher priority)

Process	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	10	3
P2	0	1	1
P3	0	2	4
P4	0	1	5
P5	0	5	2

a) Draw the Gantt chart and calculate the average waiting time and average turnaround time using
- i) FCFS scheduling
- ii) Shortest Job First (Non-preemptive)
- iii) Priority scheduling (Non-preemptive)

b) Using Round Robin scheduling with a time quantum of 2 ms,

i) Draw the Gantt chart
ii) Compute the waiting time and turnaround time for each process
iii) Find the average waiting time and average turnaround time

Solution

1. FCFS (First Come First Serve)

- Processes are executed in the order of arrival
- Non-preemptive
- Simple but causes Convoy Effect

Here the execution order becomes P1,P2,P3,P4,P5.

Gantt Chart

| P1 | P2 | P3 | P4 | P5 |
0 10 11 13 14 19

Process	TAT = CT – AT	WT = TAT − BT
P1	10	0
P2	11	10
P3	13	11
P4	14	13
P5	19	14

Average WT = (0+10+11+13+14)/5 = 9.6
Average TAT = (10+11+13+14+19)/5 = 13.4

2. SJF (Shortest Job First – Non-preemptive)

- CPU selects process with smallest burst time
- Optimal for minimum average WT
- May cause starvation

Execution Order

P2 → P4 → P3 → P5 → P1

Gantt Chart

| P2 | P4 | P3 | P5 | P1 |
0 1 2 4 9 19

Process	TAT	WT
P2	1	0
P4	2	1
P3	4	2
P5	9	4
P1	19	9

Average WT = (0+1+2+4+9)/5 = 3.2ms
Average TAT = (1+2+4+9+19)/5 = 7ms

3. Priority Scheduling (Non-preemptive)

- CPU selects process with highest priority
- Lower number = higher priority
- Can cause starvation

Execution Order

P2 → P5 → P1 → P3 → P4

Gantt Chart

| P2 | P5 | P1 | P3 | P4 |
0 1 6 16 18 19

Process	TAT	WT
P2	1	0
P5	6	1
P1	16	6
P3	18	16
P4	19	18

Average WT = (0+1+6+16+18)/5 = 8.2ms
Average TAT = (1+6+16+18+19)/5 = 12ms

4. Round Robin (Time Quantum = 2 ms)

- Each process gets fixed time slice
- Preemptive
- Used in time-sharing systems

Gantt Chart

|P1 |P2 |P3 |P4 |P5 |P1 | P5 | P1| P1|
0 2 3 5 6 8 10 12 14 19

Process	CT
P1	19
P2	3
P3	5
P4	6
P5	12

Process	TAT	WT
P1	19	9
P2	3	2
P3	5	3
P4	6	5
P5	12	7

Average WT = (9+2+3+5+7)/5 = 5.2ms
Average TAT = (19+3+5+6+12)/5 = 9ms

When comparing these algorithms, SJF is the optimal one.

Question

Consider the following set of processes with different arrival times:

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	6

P2	1	4
P3	2	2
P4	3	5

A). Using Round Robin scheduling with Time Quantum = 2 ms:
1. Draw the Gantt chart
2. Calculate Waiting Time and Turnaround Time
3. Find the average Waiting Time and Turnaround Time

B) Using Shortest Remaining Time First (SRTF) scheduling:
1. Draw the Gantt chart
2. Calculate Completion Time, Waiting Time, and Turnaround Time
3. Find the average Waiting Time and Turnaround Time

Solution

A) Gantt Chart:

P1 | P2 | P3 | P1 | P4 | P2 | P1 | P4 |

0 2 4 6 8 10 12 14 17

Process	CT	AT	BT	WT / TAT
P1	14	0	6	WT=8, TAT=14
P2	12	1	4	WT=7, TAT=11
P3	6	2	2	WT=2, TAT=4
P4	17	3	5	WT=9, TAT=14

Average Waiting Time = 6.5 ms
Average Turnaround Time = 10.75 ms

Part B) SRTF Scheduling

Gantt Chart:

When there is a tie in selecting which process with same remaining burst time to be executed, choose the one with earlier arrival time.(Here, P1 is selected over P4 first in the last step.)

P1 | P2 | P3 | P2 | P1 | P4

0 1 2 4 7 12 17

Process	CT	AT	BT	WT / TAT
P1	12	0	6	WT=6, TAT=12
P2	7	1	4	WT=2, TAT=6
P3	4	2	2	WT=0, TAT=2
P4	17	3	5	WT=9, TAT=14

Average Waiting Time = 4.25 ms
Average Turnaround Time = 8.5 ms

Multi level Feedback Queue / MLFQ Algorithm

MLFQ is a CPU scheduling algorithm that assigns processes to multiple queues with different priority levels and allows a process to move between queues based on its behaviour and CPU burst characteristics.

It is designed to favor short and interactive jobs while still allowing long CPU-bound processes to execute eventually.

Features of MLFQ

- Multiple Queues are available in MLFQ. Each queue has a different priority , time quantum and different scheduling algorithms.
- Higher priority queues have shorter time slices.
- Lower priority queues have longer time slices.
- New processes always enter the highest priority queue.
- CPU always executes the highest priority ready process.
- Time quantum: -Each queue has its own time slice.
- Demotion: If a process uses its full time slice, it is moved to a lower-priority queue.
- Promotion / Aging: If a process waits too long in a lower queue, it is promoted to the higher priority queue to prevent starvation.
- Priorities change based on process behavior (CPU-bound vs I/O-bound).

Process Aging: To prevent starvation, processes waiting too long in lower priority queues can be promoted.

Basic Rules of MLFQ

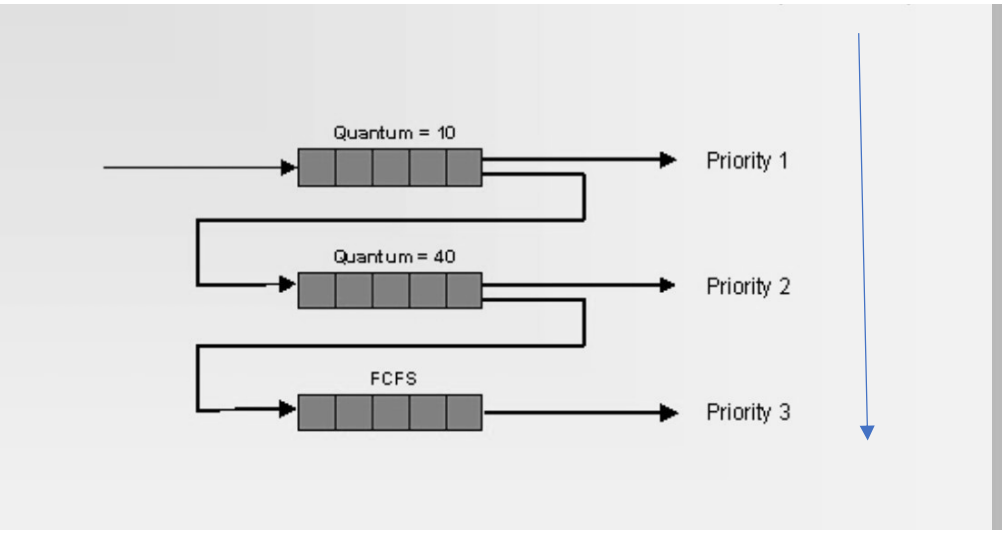
• Rule1: If Priority(process A) > Priority(processB), A runs (B doesn't).

• Rule2: If Priority(processA) = Priority(processB), A & B run in RR.

• Rule 3: When a job or a process enters the system, it is placed at the highest priority (the topmost queue). • Rule 4: Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).

• Rule 5: After some time period S, move all the jobs in the system to the topmost queue

Higher priority



Lower priority

Feedback Mechanism:

If a process uses its entire time slice, it may be demoted to a lower priority queue.

If a process yields CPU before using its quantum (I/O bound), it may stay in the same queue or move up.

Dynamic Priority: Priorities change based on process behavior (CPU-bound vs I/O-bound).

Advantages

Efficient for interactive systems.

Reduces response time for short jobs.

Balances CPU-bound and I/O-bound processes.

Disadvantages

Complex to implement.

Requires careful selection of time quanta and number of queues.

Priority inversion/starvation can occur if aging is not handled.

CPU-bound vs I/O-bound Processes

1. CPU-bound Process

A process that spends more time performing computations than waiting for I/O.

- It requires more CPU time and less I/O time.
- Uses up its full time quantum in time-sharing systems.

Example: Complex calculations, video rendering, encryption, matrix operations.

Can be demoted in MLFQ scheduling (since it is CPU-intensive).

2. I/O-bound Process

A process that spends more time waiting for I/O operations than executing on the CPU.

- CPU bursts are short, and the process often waits for disk, network, or keyboard input.
- Frequently yields CPU before using its full time quantum.

Example: Reading/writing files, web server handling requests, user interactive applications.

Can be kept at higher priority in MLFQ scheduling (since it's interactive).

Linux Completely Fair Scheduler (CFS)

CFS is the default CPU scheduler in Linux that aims to share CPU time fairly among all processes.

It doesn't use fixed time slices like traditional schedulers. Instead, it maintains a virtual runtime for each process to ensure fairness.

Features of CFS

1. Time Accounting

Each process has a virtual runtime (vruntime) that tracks how much CPU time it has received.

Goal: All runnable processes should get roughly the same share of CPU over time.

Processes that have run less get higher priority (scheduled sooner).

2. Process Selection

CFS keeps processes in a red-black tree sorted by vruntime.

The process with the lowest vruntime is picked next to run.

This ensures fair distribution and automatically favors shorter/interactive jobs.

3. Sleep and Wake-up

When a process sleeps (I/O wait), its vruntime stops increasing.

When it wakes up, it is placed in the tree according to its vruntime, ensuring it doesn't unfairly overtake other processes.

Preemption occurs if a newly awakened process has lower vruntime than the currently running process.

Case Study: Linux Scheduling Implementation

Linux Scheduling Goals

- Fair allocation of CPU time.
- Responsive system for interactive tasks.
- Efficient handling of CPU-bound tasks.

Scheduler Types in Linux:

1.Completely Fair Scheduler (CFS) – for normal tasks (SCHED_NORMAL)

Maintains a red-black tree of all runnable tasks, sorted by virtual runtime (vruntime).

The task with smallest vruntime is chosen to run next. Time slices are dynamic, based on task weight (nice value) and number of tasks. Tasks are preempted if a newly awakened task has a lower vruntime than the currently running task.

2.Real-time schedulers:

SCHED_FIFO – First-in-first-out

SCHED_RR – Round-robin

3.Deadline scheduling (SCHED_DEADLINE) – for real-time deadlines

Structures in Linux:

struct task_struct → represents each process/thread

struct rq → runqueue for each CPU

struct cfs_rq → CFS-specific runqueue

Case Study: Preemption in Linux

Preemption occurs when the currently running process is interrupted and replaced by another process that should run according to scheduling rules.

Types of Preemption:

- Voluntary Preemption – process yields CPU, e.g., schedule() called explicitly.
- Involuntary Preemption – kernel forces context switch, e.g.,

In CFS Preemption, If a task wakes up and has lower vruntime than the current task, the scheduler will preempt the current task immediately.

Case Study: Context Switching in Linux

Context Switching is a mechanism in which the CPU is switched from one process/thread to another, saving and restoring CPU state.

Steps in Context Switch

- Save current CPU state (registers, program counter) of the running process.
- Update task_struct of the old task (runtime, state, etc.).
- Select the next task to run using the scheduler (CFS or RT).
- Load CPU state of the next task.
- Resume execution of the next task.

Context Switching Functions

- schedule() – main function called when process needs to yield CPU
- context_switch() – low-level function that saves old task state and loads new task state